

ROS2 입문 2주차 요약본

6. 인터페이스 프로그래밍(응용_1)

6.1 인터페이스 프로그래밍 (ex_calculator)

- 패키지 설계
 - 계산기 개발
 - 현재 시간과 변수 a, b를 받아 연산하여 결과값 도출
 - 연산 결과값을 누적하여 목표치에 도달했을 때 이 결과값을 표시

	토픽	서비스	액션
연속성	연속성	일회성	복합(토픽 + 서비스)
방향성	단방향	양방향	양방향
동기성	비동기	동기	동기 + 비동기
다자간연결	1:1, 1:N, N:1, N:N	1:1	1:1
노드역할	Publisher, Subscriber	Server, Client	Server, Client
동작트리거	퍼블리셔	클라이언트	action 인터페이스
인터페이스	msg	srv	action
CLI명령어	ros2 topic	ros2 service	ros2 action
사용 예	센서 데이터, 로봇 상태, 로봇 좌표, 로봇 속도 명령 등	LED 제어, 모터 토크 On/Off IK/FK 계산, 이동 경로 계산 등	목적지로 이동, 물건 파지, 복합 테스트

6.2 토픽 프로그래밍

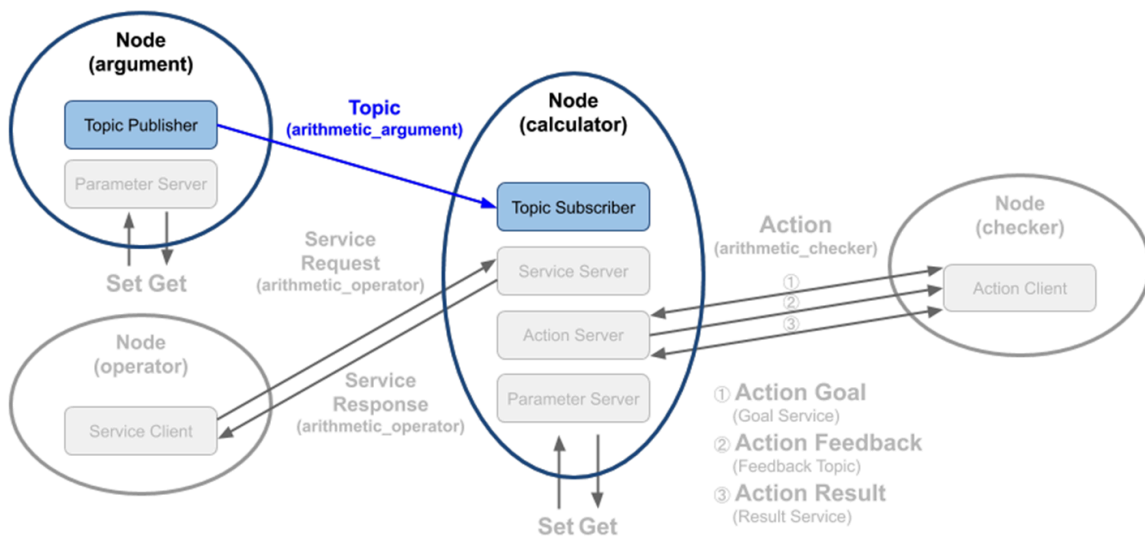
- 토픽 퍼블리셔 & 서브스크라이버

▶ 토픽 퍼블리셔 (데이터를 송신하는 프로그램)

- Node 설정
- QoS 설정
- create_publisher 설정
- 퍼블리시 함수 작성

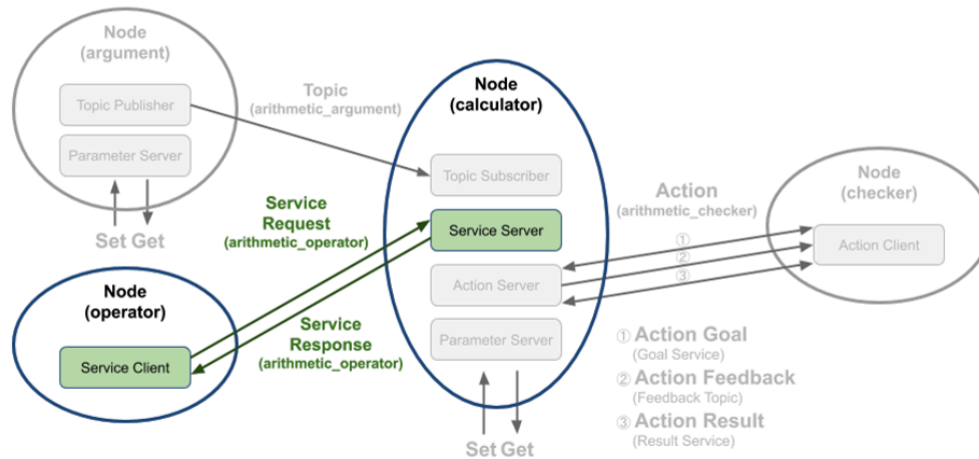
▶ 토픽 서브스크라이버 (데이터를 수신하는 프로그램)

- Node 설정
- QoS 설정
- create_subscription 설정
- 서브스크라이브 함수 작성



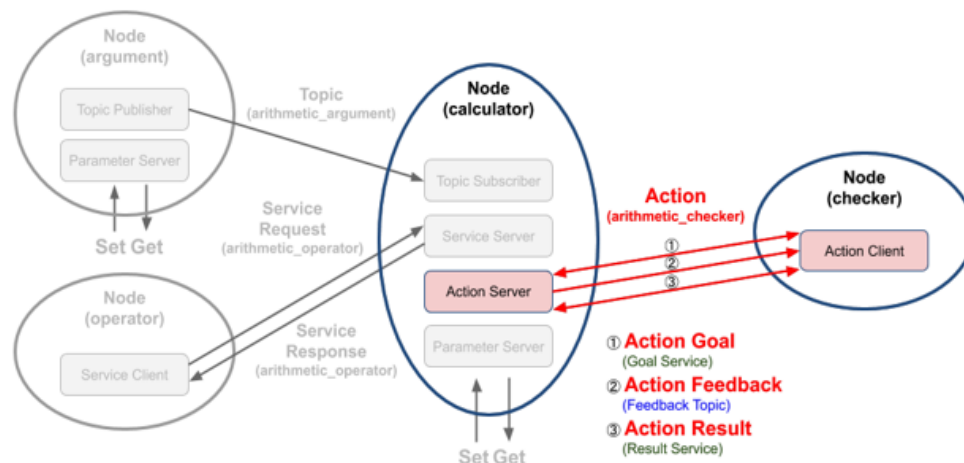
- calculator: 토픽 서브스크라이버 노드
- argument: 토픽 퍼블리셔 노드
- operator node : arithmetic_operator이라는 서비스 이름으로 calculator 노드에게 연산자(+, -, *, /)를 서비스 요청(Request) 값으로 보냄
- calculator node : 서브스크라이브하여 저장하고 있는 변수 a, b와 operator 노드로부터 요청 값으로 받은 연산자를 이용하여 계산(a 연산자 b)하고 operator 노드에게 연산의 결과값을 서비스 응답(Response) 값으로 보냄

6.3 서비스 프로그래밍



6.4 액션 프로그래밍.

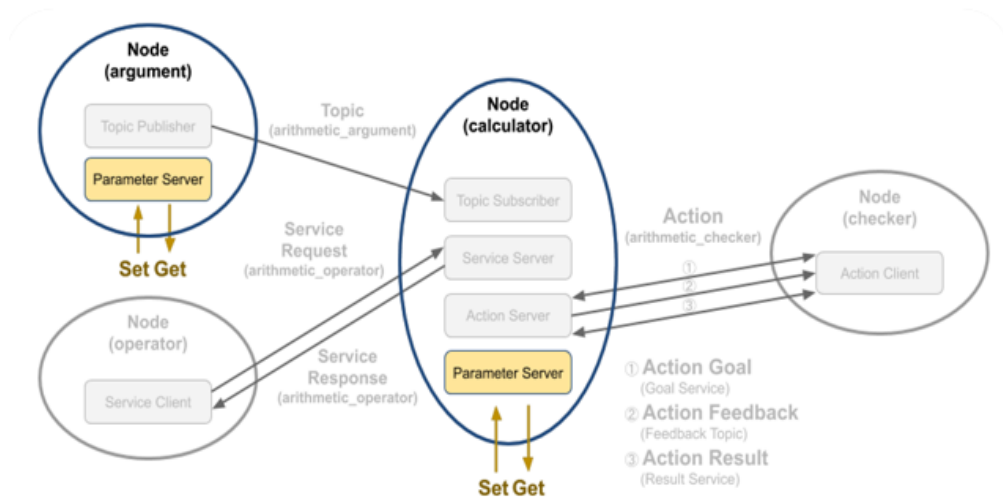
액션 목표(action goal)를 지정하는 액션 클라이언트와 액션 목표를 받아 특정 태스크를 수행하면서 중간 결과값에 해당되는 액션 피드백(action feedback)과 최종 결과값에 해당되는 액션 결과(action result)를 전송하는 액션 서버



6.5 파라미터 프로그래밍

argument 노드는 QoS 설정과 랜덤으로 생성되는 변수 a, b의 랜덤 생성 범위를 파라미터
calculator 노드는 QoS 설정을 사용

- argument 와 calculator 노드에 Parameter Server가 내장되어 있음
- 서버를 통해 외부에서 해당 노드의 설정을 동적으로 조회 하거나 변경 할 수 있다는 것을 보여줌
- 시스템을 유연하게 조정하고 런타임 중 동작을 제어하는 데 매우 유용한 기능



6.6 실행 인자 프로그래밍

- 프로그램 실행 시 추가로 입력되는 인수로, main 함수의 매개변수로 사용됨
- 실행 명령어와 함께 전달되어 프로그램 동작에 영향을 줌

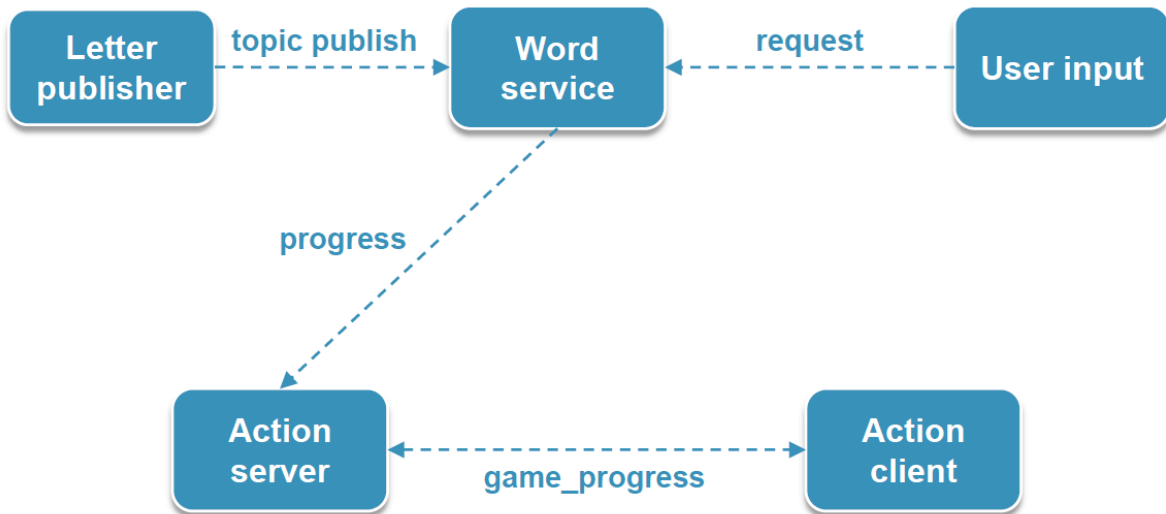
```
# -g 100은 "GOAL_TOTAL_SUM" 값을 100으로 설정
& ros2 run ex_calculator checker -g 100
```

- 실행 인자 구문
 1. 파서 만들기 (parser = argparse.ArgumentParser)
 - `ArgumentParser` 객체를 만들어 인자 파싱 준비를 함
 2. 인자 추가하기 (parser.add_argument)
 - `add_argument()` 로 실행 시 사용할 인자 이름, 타입, 기본값, 설명 등을 설정
 3. 인자 파싱하기 (args = parser.parse_args())
 - `parse_args()` 를 호출해 실제로 입력된 인자들을 파싱
 4. 인자 사용하기 (args.xxx)
 - 파싱된 인자는 `args.인자이름` 형식으로 접근하여 사용할 수 있음

7. 인터페이스 프로그래밍(응용_2)

7.1 인터페이스 프로그래밍 (hangman)

- Hangman은 단어나 문구를 추측하는 고전적인 단어 게임



```

# a부터 z까지 알파벳을 순서대로 publish
$ ros2 hangman_game letter_publisher
# 임의의 단어를 선택
$ ros2 run hangman_game word_service
# Goal 설정, action server와의 상호작용을 통해 행맨 진행 상태 업데이트
$ ros2 run hangman_game progress_action_client
# 사용자의 진행 상황을 관리, 진행상태 추적, 결과를 클라이언트에게 전달
$ ros2 run hangman_game progress_action_server
  
```

7.2 인터페이스 프로그래밍 실습

- `letter_publisher.py`
 - 타이머로 1초마다 알파벳 publish

```
self.publisher_.publish(String(data=self.alphabet[self.index]))
```

- `word_service.py`
 - `letter_topic` 수신 → 현재 글자로 `CheckLetter` 서비스 요청 응답 처리
 - 게임 상태는 `Progress` 메시지로 퍼블리시

- `user_input.py`
 - 사용자 입력 감지 후 `CheckLetter` 서비스 요청 전송

```
self.cli.call_async(request)
```

- `progress_action_client.py`
 - 목표 전송 후 피드백과 결과를 처리

```
self._action_client.send_goal_async(goal_msg, feedback_callback=self.feed  
back_callback)
```

- `progress_action_server.py`
 - `progress` 토픽 수신 → 피드백으로 클라이언트에게 전달
 - `execute_callback()` 에서 결과 반환

8. ROS2 응용

8.1 ROS2 bag

- ROS2 BAG은 시스템의 토픽에 게시된 데이터를 기록하기 위한 플랫폼 기능
- 각 토픽에서 수집된 데이터를 데이터베이스에 저장하며, 이를 재생하여 테스트 및 실험 결과를 재현할 수 있음
- ROS2 BAG(Turtlesim)
 - Turtle의 움직임을 BAG 파일에 기록
 - 기록된 BAG 파일을 이용하여 turtle 움직이기

```
# 첫번째 터미널에서 turtlesim_node 실행
$ ros2 run turtlesim turtlesim_node
```

```
# 두번째 터미널에서 turtle_teleop_key 실행
$ ros2 run turtlesim turtle_teleop_key
```

```
# 세번째 터미널에서 /turtle1/cmd_vel 토픽 기록
• /turtle1/cmd_vel이라는 이름을 가진 토픽을 turtle_bag이라는 이름의 BAG파일에 기록
$ ros2 bag record -o turtle_bag /turtle1/cmd_vel

# 레코딩
• 두번째 터미널로 이동하여 방향키를 사용해 turtle을 임의로 이동시키기
$ ros2 run turtlesim turtle_teleop_key

# 세번째 터미널에서 실행되고 있는 record 종료
• Ctrl + c (종료)

# 세번째 터미널에서 저장된 BAG 파일 확인
$ ros2 bag info turtle_bag

# 첫번째 터미널에서 실행중인 turtlesim을 종료 후 재실행
$ ros2 run turtlesim turtlesim_node

# 세번째 터미널에서 다음 명령어를 이용하여 저장되어 있는 BAG 파일 재생
• turtle의 궤적이 기록 당시와 정확히 일치하지 않을 수 있음
• 각 명령이 완전히 완료된 후 다음 명령을 실행해야함
$ ros2 bag play turtle_bag
```

8.2 ROS2 Cli

- 2D 카메라

```
# BAG 파일에 기록된 2d 카메라 정보 불러오기
• 링크에서 다운로드 받은 후 압축 풀기
tar -zxvf rosbag2_video.tar.gz

# bag 파일의 정보 확인
$ ros2 bag info rosbag2_video

# bag 파일 반복 재생
$ ros2 bag play rosbag2_video --loop
```

```
# 새로운 터미널에서 rviz 실행
$ rviz2
```

```
# Rviz에서 add버튼을 누른 후 Image 선택 후 OK 버튼 누르기
```

```
# 사이드바에서 이미지의 topic name을 "\video_frames"로 바꾸기
# 하단의 Image 창에 영상이 재생되는 것을 확인
```

- 3D 카메라

```
# BAG 파일에 기록된 3d 카메라 정보 불러오기
```

- 링크에서 다운로드 받은 후 압축 풀기

```
tar -zxvf realsense.tar.gz
```

```
# bag 파일의 정보 확인
```

```
$ ros2 bag info realsense
```

```
# bag 파일 반복 재생
```

```
$ ros2 bag play realsense --loop
```

```
# 새로운 터미널에서 rviz 실행
```

```
$ rviz2
```

```
# BAG 파일에 기록된 3d 카메라 정보 불러오기
```

- FixedFrame → camera_depth_optical_frame으로 바꾸기
- ADD → By topic → camera → camera → depth → color → points → PointCloud2 선택 후 OK 버튼 클릭

- Lidar

```
# BAG 파일에 기록된 lidar 정보 불러오기
```

- 링크에서 다운로드 받은 후 압축 풀기

```
tar -zxvf race_car.tar.gz
```



```
# bag 파일의 정보 확인
$ ros2 bag info race_car/

# bag 파일 반복 재생
$ ros2 bag play race_car --loop

# 새로운 터미널에서 rviz 실행
$ rviz2

# BAG 파일에 기록된 lidar 정보 불러오기
• FixedFrame → luminar_front으로 바꾸기
• ADD → by topic → vehicle8 → luminar_front_points → PointCloud2 선택 후
OK 버튼 클릭
• 사이드 바의 PointCloud2에서 Size(m)을 0.1로 변경
```

8.3 Python으로 토픽 구독

```
# Vscode 실행
$ code .

# turtlesim_node를 실행
$ ros2 run turtlesim turtlesim_node

# Vscode에서 jupyter notebook 실행
import rclpy as rp
from turtlesim.msg import Pose

# topic list 조회
$ ros2 topic list -t

# rclpy의 초기화 및 '/sub_test' 노드 생성
rp.init()
test_node = rp.create_node('sub_test')

# 터미널에서 '/sub_test'노드가 생성되었음을 확인
```

```
$ ros2 node list
```

```
# Subscription에서 실행할 callback 함수 작성
```

```
def callback(data):  
    print("————→")  
    print("/turtle/pose : ", data)  
    print("X : ", data.x)  
    print("Y : ", data.y)  
    print("Theta : ", data.theta)
```

```
# 토픽 subscriber 만들기
```

```
test_node.create_subscription(Pose, 'turtle1/pose', callback, 10)
```

```
# test_node 구독
```

```
rp.spin_once(test_node)
```

8.4 Python으로 토픽 발행

```
# Vscode 실행
```

```
$ code .
```

```
# turtlesim_node를 실행
```

```
$ ros2 run turtlesim turtlesim_node
```

```
# Vscode에서 jupyter notebook 실행
```

```
import rclpy as rp
```

```
from geometry_msgs.msg import Twist
```

```
rp.init()
```

```
test_node = rp.create_node('pub_test')
```

```
# Twist 선언
```

```
msg = Twist()
```

```
print(msg)
```

```
# x축 선형 속도를 2.0으로 설정한 후, 발행할 메시지로 준비
```

```

msg.linear.x = 2.0
print(msg)

# 토픽 발행
pub = test_node.create_publisher(Twist, '/turtle1/cmd_vel', 10)
pub.publish(msg)

# 추가 동작도 가능
msg.linear.x = 2.0
msg.angular.z = 2.0
pub.publish(msg)

# Timer를 이용해 토픽 발행
cnt = 0
def timer_callback():
    global cnt
    cnt += 1

    print(cnt)
    pub.publish(msg)

    if cnt > 3:
        raise Exception('Publisher Stop')

# 2초마다 timer_callback 함수 실행
timer_period = 2.0
timer = test_node.create_timer(timer_period, timer_callback)
rp.spin(test_node)

# 노드 종료 시키기
test_node.destroy_node()

```

8.5 Python으로 서비스 클라이언트

```

# Vscode 실행
$ code .

```

```

# turtlesim_node를 실행
$ ros2 run turtlesim turtlesim_node

# 모듈 import
import rclpy as rp
from turtlesim.srv import TeleportAbsolute

rp.init()
test_node = rp.create_node('client_test')

# 클라이언트를 생성
service_name = '/turtle1/teleport_absolute'
cli = test_node.create_client(TeleportAbsolute, service_name)

# request 객체를 생성
req = TeleportAbsolute.Request()
req

# x좌표를 1.0, y좌표를 1.0, theta(회전각)를 3.14
req.x = 1.
req.y = 1.
req.theta = 3.14

req

# req의 x성분만 3.0으로 바꾼 후 call_async를 통해 서비스 호출
req.x = 3.

cli.call_async(req)
rp.spin_once(test_node)

# test_node에서 스핀을 실행
req.y = 9.

while not cli.wait_for_service(timeout_sec=1.0):
    print("Waiting for service")

```

```
cli.call_async(req)
rp.spin_once(test_node)

# 노드 종료 시키기
test_node.destroy_node()
```

8.6 Python으로 액션 서버

```
# Vscode 실행
$ code .

# turtlesim_node를 실행
$ ros2 run turtlesim turtlesim_node

# action_server.ipynb파일 생성
# TurtleRotateServer 클래스 정의
# execute_callback() 메서드 정의
• 명령을 실행하고, 10초 동안 남은 진행 상태를 피드백으로 클라이언트에 전달
  실행 도중 취소 요청이 들어오면 작업을 취소하고, 성공적으로 완료되면 결과를 반환

# 액션 서버 초기화 및 인스턴스 생성
rclpy.init()
server = TurtleRotateServer()

# 액션 서버 실행
executor = MultiThreadedExecutor()
rclpy.spin(server, executor=executor)

# 액션 서버 종료
```

8.7 Python으로 액션 클라이언트

```
# 필요한 모듈 import (Ros2와 액션 메시지)
# TurtleRotateClient 클래스 주요 메서드
• goal_response_callback: 서버에서 각도 정보를 수신했는지 확인
```

- `feedback_callback`: 액션 실행 중 발생하는 피드백 로그를 기록
- `result_callback`: 액션 완료 후 결과를 처리하고 ROS 노드를 종료

액션 클라이언트 초기화 및 인스턴스 생성

```
rclpy.init()
```

```
client = TurtleRotateClient()
```

액션 클라이언트 실행 후 종료

```
client.send_goal(3.14)
```

```
rclpy.spin(client)
```

액션 클라이언트 실행

- 회전하는 도중에 지속적으로 중간 실행 결과를 피드백 받을 수 있음

9. ROS 응용 핵심정리

9.1 Topic

- 비동기식 단방향 메시지 송수신 방식 (Publisher → [Topic] → Subscriber)
- 1:1 통신을 기본으로 함 (1 : N, N : N, N : 1도 가능)
- 비동기성과 연속성을 가지기에 센서 값 전송 및 항시 정보를 주고받아야 하는 부분에서 주로 쓰임

9.2 Service

- 동기식 양방향 메시지 송수신 방식 (Service Client ↔ Service Server)
- msg 인터페이스의 변형인 srv 인터페이스를 사용
- 동일한 서비스 서버에 대해 복수의 클라이언트를 가질 수 있도록 설계됨

9.3 Action

- 비동기식, 동기식 양방향 메시지 송수신 방식
- 액션 목표, 피드백, 결과 데이터는 msg, srv 인터페이스의 변형인 action인터페이스를 사용

- 액션 목표 전달 이후 액션의 실행값을 액션 클라이언트에게 전달할 수 있어서 비동기, 동기 방식이 혼재된 액션의 처리를 보다 원활하게 해줌

9.4 인터페이스(interface)

- 토픽, 서비스, 액션에서 사용되는 데이터 타입
- 토픽 - msg 파일, 서비스 - srv 파일, 액션 - action 파일
- 미리 선언된 인터페이스를 바로 사용 가능하나, 필요에 따라 커스텀 인터페이스 생성 가능
- 여러 개의 패키지를 가지는 경우, 별도의 인터페이스 패키지를 생성하여 사용하는 것을 추천

9.5 VScode - extension

항목	Service Client	Action Client
목적	요청-응답 1회성 호출	장시간 작업의 관리
단계 수	1단계 (call_async)	3단계 (send_goal_async → GoalHandle → get_result_async)
중간 상태	없음	진행 상태(feedback), 취소 등 처리 가능
.result() 위치	call_async 후 future에서	get_result_async 후 future에서
피드백 처리	불가	가능 (feedback_callback)
쓰임새	빠른 계산 요청 (예: IK)	장기 제어 작업 (예: 궤적 추종)